

⚠ Please do not reach out to reviewers directly; messaging them will not speed up your review.

What Makes a Good Task

Creating high-quality tasks is both an art and a science. This guide covers what separates excellent tasks from mediocre ones.

The Core Principle

A good task is one that an expert human can solve confidently, but that challenges or stumps current AI coding agents.

We're not looking for trivia or trick questions. We want genuine engineering challenges that require:

- Multi-step reasoning
- Domain expertise
- Practical problem-solving skills

Key Requirements

1. Difficulty Target

The worst-performing model's accuracy must be $\leq 80\%$ across GPT-5.5 and Claude Opus 4.8.

DIFFICULTY	THRESHOLD
Hard	Accuracy $\leq 20\%$ on the best model, OR $\leq 20\%$ on the worst model

DIFFICULTY	THRESHOLD
Medium	20% < accuracy ≤ 60% on the worst model
Easy	60% < accuracy ≤ 80% on the worst model

Tasks where the worst model scores above 80% will NOT be accepted. See [Difficulty Guidelines](#).

2. Multi-Step Complexity

Tasks must require chaining multiple commands, handling intermediate states, and reasoning. Single-command tasks are too easy.

Good: "Debug the failing test suite, fix the three bugs causing failures, and verify all tests pass."

Bad: "Run the test suite." (Too simple)

3. Clear & Unambiguous

The task must be fully specified. An agent should understand exactly what to do without guessing.

Good:

"Implement a function `find_longest_palindrome(s: str) -> str` that returns the longest palindromic substring. If there are ties, return the first one."

Bad:

"Write code for palindromes." (Ambiguous)

4. Testable & Verifiable

Every task must have deterministic tests that verify completion.

PYTHON

 Copy

```
def test_longest_palindrome():
    assert find_longest_palindrome("babad") in ["bab", "aba"]
    assert find_longest_palindrome("cbbd") == "bb"
    assert find_longest_palindrome("") == ""
```

5. No Cheating Opportunities

Think like a malicious agent. Ensure they cannot:

- Look inside test files for answers
- Edit data files to pass tests
- Delete tests to avoid failures
- Hardcode expected outputs

How to Make Tasks Harder

Debugging-Style Tasks

When an agent must figure out the root cause of an issue, it inherently requires reasoning.

Niche Knowledge

Use publicly available but rarely-trained knowledge. Example:

[Blockchain/NFT task](#)

Bespoke Rules

A custom rule buried among common rules confuses agents. Example:

[Bespoke rule PR](#)

Multi-Step Tasks

Each step has a chance of failure, making overall failure rate higher.

What to Avoid

ANTI-PATTERN	WHY IT'S BAD
Trivia questions	Tests memorization, not reasoning
Ambiguous requirements	Agents can't know what's expected
External dependencies	Requires API keys, network, etc.
Simple one-liners	Agent solves instantly
Brittle tests	String matching, hardcoded values

Quality Checklist

Before submitting, verify:

- ☐ Problem statement is clear and complete
- ☐ Difficulty is < 80% pass rate
- ☐ Multi-step reasoning required
- ☐ All constraints explicitly stated
- ☐ Test cases cover all requirements
- ☐ No cheating opportunities
- ☐ Human-written (not LLM-generated instruction.md)

Next Steps

- [Understand task components](#)
- [See the task taxonomy](#)
- [Review example tasks](#)

PREVIOUS

[After Submission](#)

NEXT

[Task Components](#)